ARTA Portal VAPT Report

Target: ARTA Portal codebase and reachable local HTTP endpoints
Workspace: c:\xampp\htdocs\Arta-Portal
Assessment date: 2026-05-31
Method: targeted static review plus limited non-destructive HTTP validation on a local PHP server

Executive summary

This assessment confirmed multiple exploitable weaknesses affecting authentication, password recovery, access-control administration, and data exposure. The most severe issue is that role-management endpoints are reachable without authentication and allow unauthenticated creation and deletion of roles. A second high-risk issue allows the password-reset flow to directly overwrite a user password and send a new plaintext password by email. Additional assignment endpoints expose user directory data without a session, and many APIs return raw internal exception details to clients.

Risk Summary

Severity	Count	Notes
CRITICAL	1	Privilege model tampering via unauthenticated role administration
HIGH	3	Password reset abuse, anonymous data exposure, broad exception leakage
MEDIUM	2	Broken reset flow logic and CSRF/session-hardening gaps

Confirmed Findings

CRITICAL Unauthenticated role-management APIs allow privilege model tampering

Affected endpoints: `/api/role/get_roles.php`, `/api/role/create_role.php`, `/api/role/delete_role.php`

Code evidence:

- `api/role/create_role.php:6` only checks the HTTP method before allowing inserts.
- `api/role/create_role.php:38` inserts directly into `roles_tbl`.
- `api/role/delete_role.php:6` only checks the HTTP method before allowing deletes.
- `api/role/delete_role.php:48` deletes directly from `roles_tbl`.

HTTP validation: Confirmed over `http://127.0.0.1:8088` without any session cookie. A temporary role was successfully created and deleted using direct POST requests.

Impact: An unauthenticated attacker can modify the application's role catalog, potentially enabling unauthorized access paths, destabilizing assignment workflows, or preparing privilege-abuse chains.

Recommendation: Require authenticated admin authorization on all role endpoints, add CSRF protection for browser-initiated requests, and log privileged changes with actor identity and source IP.

HIGH Forgot-password flow directly resets passwords and sends plaintext credentials by email

Affected endpoint: `/api/auth/forgot_password.php`

Code evidence:

- `api/auth/forgot_password.php:53` returns a distinct `Email not registered` response, enabling account enumeration.
- `api/auth/forgot_password.php:60` generates a replacement password server-side.
- `api/auth/forgot_password.php:66` immediately updates the stored password before user confirmation.
- `api/auth/forgot_password.php:114` and `:120` return detailed backend and mailer errors to the client.
- `api/auth/forgot_password.php:6` uses wildcard CORS, widening the attack surface for scripted abuse.

Impact: Anyone who can trigger the endpoint for a valid account can force a password change, interrupt user access, and cause credentials to be transmitted in plaintext through email. The endpoint also leaks whether an account exists.

Recommendation: Replace this with a tokenized reset-link flow, return a generic response for both existing and non-existing emails, remove plaintext password distribution, rate-limit the endpoint, and suppress internal exception details from client responses.

HIGH Assignment lookup APIs expose user directory data without authentication

Affected endpoints: `/api/assignment/get_users.php`, `/api/assignment/get_filter_values.php`

Code evidence:

- `api/assignment/get_users.php:3` loads the DB connection but performs no session or role check.
- `api/assignment/get_users.php:25` queries user names and office/division/unit/designation metadata.
- `api/assignment/get_users.php:38` returns the dataset directly as JSON.
- `api/assignment/get_filter_values.php:3` also lacks session validation.
- `api/assignment/get_filter_values.php:24` returns internal organization values directly.

HTTP validation: Confirmed over `http://127.0.0.1:8088` without a session. The endpoint returned real user records and office names anonymously.

Impact: Anonymous users can enumerate personnel names and organization structure. This materially lowers the effort for phishing, password spraying, and targeted social engineering.

Recommendation: Require authenticated and authorized users for these endpoints, minimize returned fields, and implement rate-limiting and audit logging.

HIGH Internal exception details are exposed across multiple APIs

Affected areas: registration, forgot-password, assignment, systems, office/division/unit/designation, list, and test/diagnostic APIs

Representative evidence:

- `api/auth/register.php:139` and `:146` return database and server exception messages.
- `api/auth/forgot_password.php:114` and `:120` return DB and email-sending errors verbatim.
- `api/assignment/assign_system_subadmin.php:51` returns raw PDO messages.
- `api/assignment/bulk_assign_system_subadmin.php:68` returns raw PDO messages.
- `api/assignment/get_assigned_systems.php:48` and `:52` leak backend error details.
- `api/test_connection.php:73` exposes `error_details`.

Impact: Error leakage can reveal schema names, query structure, environment behavior, and integration details useful for follow-on attacks.

Recommendation: Standardize on generic client-facing errors, keep detailed diagnostics in server logs only, and remove any diagnostic/test endpoints from production exposure.

MEDIUM Reset-password logic is inverted, causing valid reset tokens to fail

Affected endpoint: `/api/auth/reset_password.php`

Code evidence:

- `api/auth/reset_password.php:33` loads candidate reset tokens.
- `api/auth/reset_password.php:46` checks `if ($userId)`.
- `api/auth/reset_password.php:47` then returns `Invalid or expired token` when a valid token match has actually been found.

Impact: Valid password reset requests are rejected, resulting in broken recovery and potential lockout scenarios. In practice, users are pushed toward the weaker forgot-password flow above.

Recommendation: Invert the condition to reject only when no matching token is found, then expire the token atomically after successful password update.

MEDIUM CSRF and session hardening are inconsistent across state-changing endpoints

Code evidence:

- `api/auth/get_csrf.php` issues a registration token, but the reviewed registration and role-management write endpoints do not verify a CSRF token.
- `includes/config_access.php` starts sessions without explicit `cookie_secure`, `cookie_httponly`, or `cookie_samesite` controls.
- `api/auth/login.php` regenerates the session ID after login, which is good, but that protection is not consistently paired with broader cookie hardening in the shared config layer.

Impact: Browser-based authenticated users remain more exposed to cross-site request forgery and weaker session cookie protections than necessary.

Recommendation: Enforce CSRF validation on every browser-reachable state-changing endpoint and centralize secure session cookie settings in the shared bootstrap.

Positive Security Controls Observed

- The active SSO validator in `api/sso/validate_token.php` now enforces expiry and one-time-use semantics.
- `api/auth/login.php` uses password hashing and regenerates the session ID after successful login.
- The newer role assignment endpoints validate submitted roles against `roles_tbl` before insert.

Recommended Remediation Order

1. Immediately protect all `/api/role/` write endpoints with authenticated admin authorization.
2. Replace the current forgot-password design with tokenized reset links and generic responses.
3. Add authorization checks to assignment lookup endpoints and other data-returning APIs that currently trust anonymous callers.

4. Remove raw exception details from client responses and disable `display_errors` on exposed APIs.
5. Fix the reset-password token validation bug and enforce CSRF/session hardening consistently.

Assessment Notes

This report is limited to the code and local endpoints available in the current workspace. It is not a full production network penetration test and does not include infrastructure scanning, authenticated browser crawling, or destructive exploitation.

Post-fix Addendum

On 2026-05-31, the following high-priority remediation work was completed in the Portal codebase:

- Role-management endpoints were restricted to authenticated admin or subadmin sessions.
- Assignment lookup endpoints that exposed users and organization metadata were restricted to authenticated admin or subadmin sessions.
- The forgot-password flow was converted from direct password replacement to one-hour reset links.
- The reset-password endpoint logic was corrected so valid tokens succeed and are marked used atomically.
- Shared session bootstrap was hardened with stricter cookie settings.

Portal to AMS SSO verdict

Portal-side SSO handoff is functioning with the preserved `/sso-login?token=...` AMS contract.

- A fresh Portal token validated successfully through `/api/sso/validate_token.php`.
- A fresh Portal token sent directly to AMS at `http://127.0.0.1:8089/sso-login?token=...` produced an internal AMS redirect to the expected role path instead of a Portal error.
- An AMS `500` was reproduced only when the token had already been consumed, and AMS surfaced that validator failure as `{"status":"error","message":"Token validation failed (HTTP error)","http_status":409}`.

Conclusion: the current Portal implementation does not require a contract change for AMS. If AMS still shows a server error during real redirects, the likely AMS-side adjustment is to handle non-200 validator responses such as `409 Token already used`, `410 Token expired`,

and `403 Unauthorized` as controlled login or retry failures instead of converting them into a generic `500`. AMS should also ensure it validates a token only once per login attempt and does not accidentally replay the same token during middleware or redirect chains.

Generated locally for the ARTA Portal workspace on 2026-05-31.